

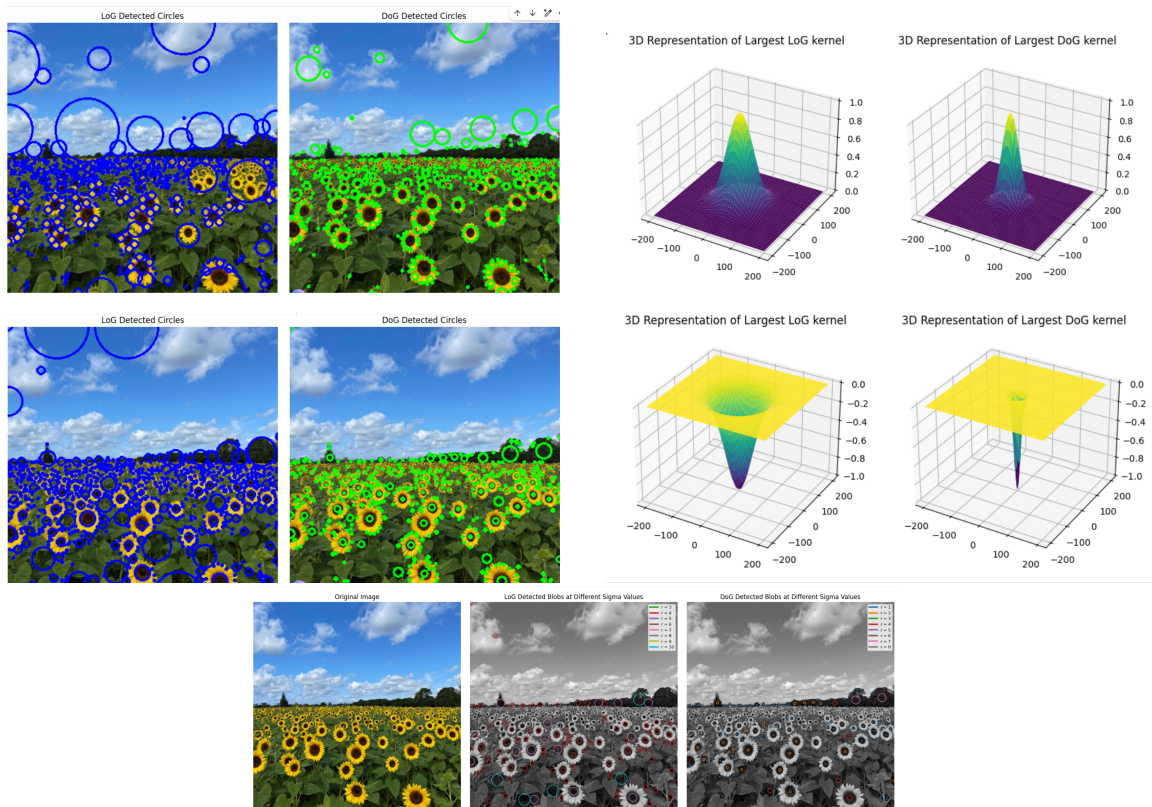
Assignment 2: Alignment and Fitting

Name: T.P.N.S Peiris
 Index No.: 220454P
 Submission Date: 07 th August 2025
 GitHub: Assignment 02

1 Blob Detection

In this task, I compared the performance of Laplacian of Gaussian (LoG) with Difference of Gaussians (DoG) for detecting white and dark blobs at different r values. The larger blobs were successfully detected with r values of 9 and 10, whereas smaller blobs were detected with r values of 1, 2, and 3. LoG and DoG kernels were configured to detect local maxima, effectively identifying brighter regions like sunflower petals while the kernels were adapted to detect local minima focusing on darker regions like the sunflower centers. LoG performed better for larger blobs while DoG performed better for smaller blobs.

```
blobs_log=blob_log(gray,max_sigma=30,num_sigma=10,threshold=0.1)
blobs_dog = blob_dog(gray, max_sigma=30, threshold=0.1)
```



Largest circle using LoG for brighter blobs: Radius= 42.43, Largest circle using DoG for brighter blobs: Radius=26.84, Largest circle using LoG for darker blobs: Radius=42.43, Largest circle using DoG for darker blobs: Radius=10.49

2 RANSAC Line fitting and Circle fitting

```
# RANSAC for line
def ransac_line(X, threshold=0.3, num_iter=100, min_inliers=30):
    best_model = None
    best_inliers = []
```

```

for _ in range(num_iter):
    sample_points = sample(list(X), 2)
    model = fit_line_from_points(*sample_points)
    distances = line_distance(model, X)
    inliers = X[distances < threshold]
    if len(inliers) > len(best_inliers):
        best_model = model
        best_inliers = inliers

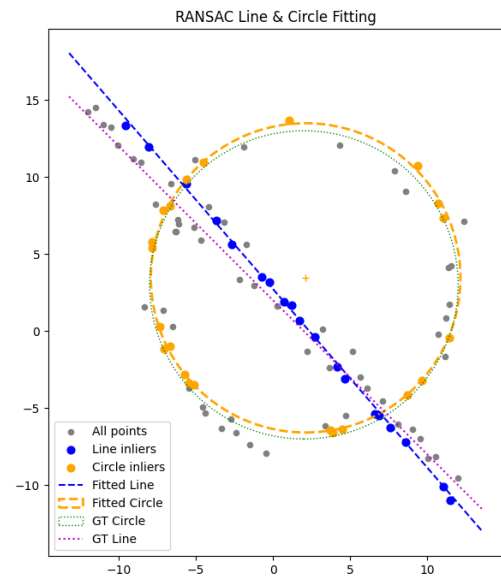
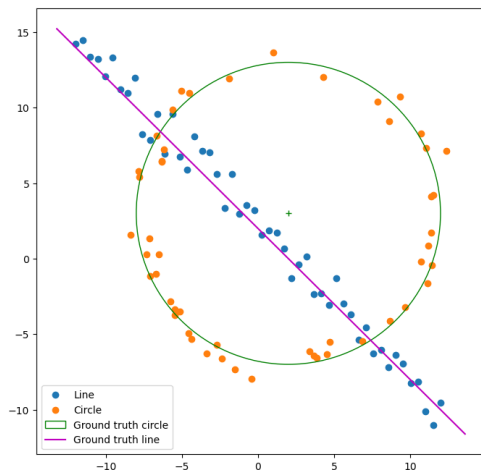
return best_model, best_inliers

# RANSAC for circle
def ransac_circle(X, threshold=0.3, num_iter=100, min_inliers=30):
    best_model = None
    best_inliers = []

    for _ in range(num_iter):
        sample_points = sample(list(X), 3)
        try:
            model = fit_circle_from_3_points(*sample_points)
        except np.linalg.LinAlgError:
            continue
        errors = circle_radial_error(model, X)
        inliers = X[errors < threshold]
        if len(inliers) > len(best_inliers):
            best_model = model
            best_inliers = inliers

    return best_model, best_inliers

```



If we fit the circle first, the 3 randomly chosen points might include points from the line, which will distort the estimated circle. We will find less inliers, and the fit will be wrong. This will also remove valid line points and ruin the next step.

3 Superimposing an Image on Another

The goal of this task is to superimpose an image onto another image by estimating a homography between defined points, followed by blending them with weighted addition.

```

# Define source and destination points
h_flag, w_flag = flag.shape[:2]

src_points = np.array([
    [0, 0],           # top-left of flag
    [w_flag, 0],      # top-right
    [w_flag, h_flag], # bottom-right
    [0, h_flag]       # bottom-left
], dtype=np.float32)

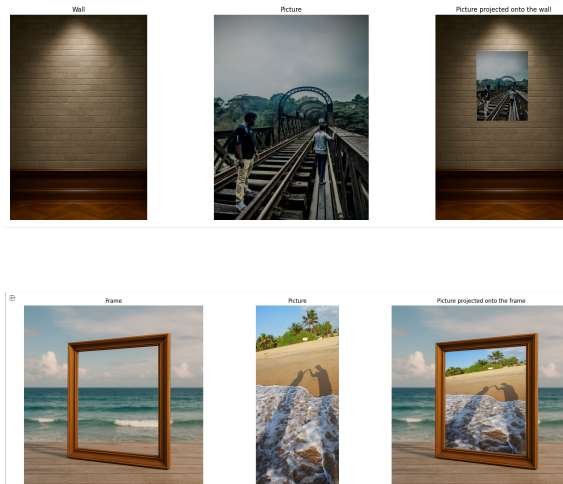
dst_points = np.array([
    [303, 273],
    [691, 273],
    [687, 793],
    [310, 793]
], dtype=np.float32)

# Compute homography and warp flag
H, _ = cv2.findHomography(src_points, dst_points)
warped_flag = cv2.warpPerspective(flag, H, (building.shape[1],
    building.shape[0]))

# Create mask and blend
mask = np.any(warped_flag != 0, axis=2).astype(np.uint8) * 255
mask_inv = cv2.bitwise_not(mask)

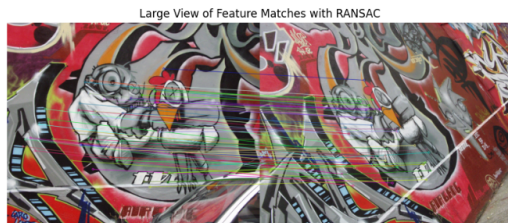
bg = cv2.bitwise_and(building, building, mask=mask_inv)
fg = cv2.bitwise_and(warped_flag, warped_flag, mask=mask)
result = cv2.add(bg, fg)

```



4 Image Stitching

I performed SIFT feature matching between img1.ppm and img5.ppm and used FLANN based matcher and match the descriptors using K nearest neighbour method. After this step homography was computed to stitch the two images.



Computed Homography Matrix:
[[6.40934900e-01 6.19857131e-01 -1.79431528e+01]
[1.42122102e-01 9.01963700e-01 1.56878755e+02]
[4.14239845e-04 -1.15245877e-04 1.00000000e+00]]
Actual Homography Matrix from Dataset:
[[6.63789505e-01 6.8808334e-01 -3.1238335e+01]
[1.4495500e-01 9.7128304e-01 1.4877428e+02]
[4.2518504e-04 -1.3930359e-05 1.0000000e+00]]
Error between Computed and Actual Homography:
15.564124802304079



```
sift = cv.SIFT_create()
keypoints1, descriptors1 = sift.detectAndCompute(img1, None)
keypoints5, descriptors5 = sift.detectAndCompute(img5, None)
flann_index_kdtree = 1
index_params = dict(algorithm=flann_index_kdtree, trees=5)
search_params = dict(checks=100)
flann = cv.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(descriptors1, descriptors5, k=2)
max_inliers = 0
best_homography = None
threshold = 2.0 # Threshold for considering inliers
iterations = 2000 # Number of iterations for RANSAC
src_pts = np.float32([keypoints1[m.queryIdx].pt for m in good_matches]).reshape(-1, 2)
dst_pts = np.float32([keypoints5[m.trainIdx].pt for m in good_matches]).reshape(-1, 2)
for _ in range(iterations):
    # Randomly select 4 points
    idx = random.sample(range(len(good_matches)), 4)
    src_sample = src_pts[idx]
    dst_sample = dst_pts[idx]
    H, _ = cv.findHomography(src_sample, dst_sample, 0)
    if H is None:
        continue
    projected_pts = cv.perspectiveTransform(src_pts.reshape(-1, 1, 2), H).reshape(-1, 2)
    distances = np.linalg.norm(dst_pts - projected_pts, axis=1)
    inliers = np.sum(distances < threshold)
    if inliers > max_inliers:
        max_inliers = inliers
        best_homography = H
if best_homography is not None:
    projected_pts = cv.perspectiveTransform(src_pts.reshape(-1, 1, 2), best_homography).reshape(-1, 2)
    inliers_mask = np.linalg.norm(dst_pts - projected_pts, axis=1) < threshold
    img_ransac_matches = cv.drawMatches(img1, keypoints1, img5, keypoints5, good_matches, None, mat
if best_homography is not None:
    height, width, _ = img5.shape
    stitched_img = cv.warpPerspective(img1, best_homography, (width, height))
```